

Python Radio 56: Crystal Radio

A Tool to Help Build Power-Free Receivers



[Simon Quellen Field](#)

Follow

8 min read

.

2 days ago

Listen

Share

More

Press enter or click to view image in full size



MidJourney

One of my hobbies is building crystal radios.

There is something fascinating about pulling signals out of the air without using any batteries.

Sometimes I try to build them using as few modern parts as possible. I wind my own coils, build my own capacitors, make diodes out of crystals or rusty metal, and make my own headphones. Aiming for a steampunk look.

Other times, I try to optimize them for the best reception, the loudest audio, or the best tuning.

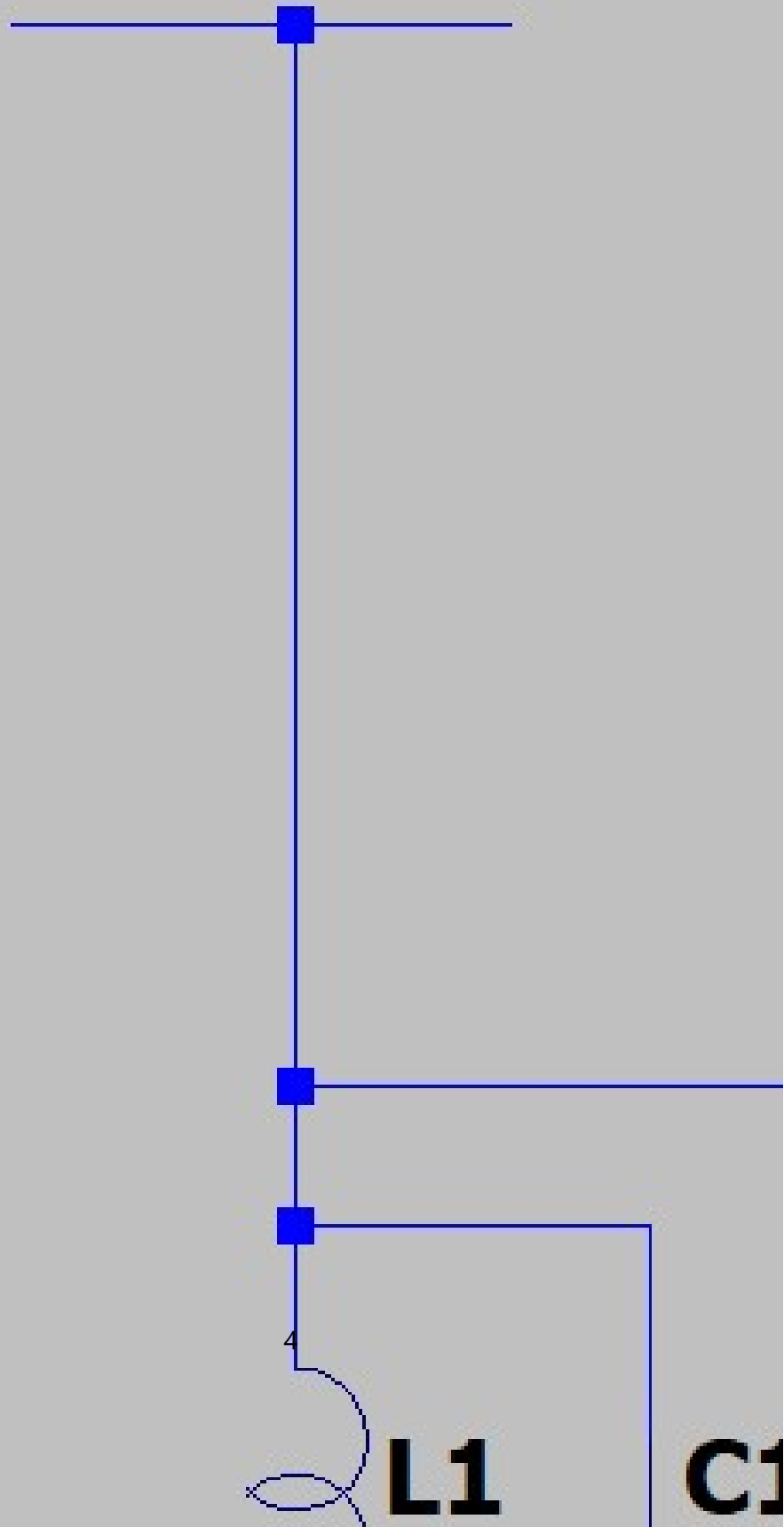
But it is also fun to use high-tech toys to build them. My oscilloscope, my multimeter, my big server computer, or my Raspberry Pi Pico 2 (the RP2350).

That last item is the centrepiece of today's project.

Crystal Radios

A crystal radio can be very simple:

Antenna



Crystal Radio Schematic by the author

The parts L1 and C1 are how we tune to the frequency we want. We can remove them, and we would hear all the stations at once. If there is only one station strong enough to power the earphones, that may be fine.

The Mechanism of Tuning

These two parts form a “tank circuit.” The tank circuit resonates at the frequency of interest.

When power comes in through the antenna, the coil conducts it to ground. As it does, a magnetic field builds up around the coil. When the power cycles to zero, the magnetic field collapses and generates a current. This current charges up the capacitor.

Then the radio frequency current cycles negative, building up the magnetic field in the opposite direction. The capacitor discharges into the coil.

The current sloshes from the coil to the capacitor and back, like waves in a bathtub. The frequency depends on how many coils there are in the inductor, and how big the plates are (or how close they are) in the capacitor.

At its resonant frequency, the tank circuit no longer allows any current to get from the antenna to ground. Other frequencies pass through just fine.

The antenna has another path to ground, however. It can pass through the diode and the earphone to get there. This produces sound. The diode removes the negative half of the radio wave, leaving

only the audio portion to get to our ears.

Building our own coil and capacitor presents us with a problem. How can we tell if they combine to have the right resonant frequency?

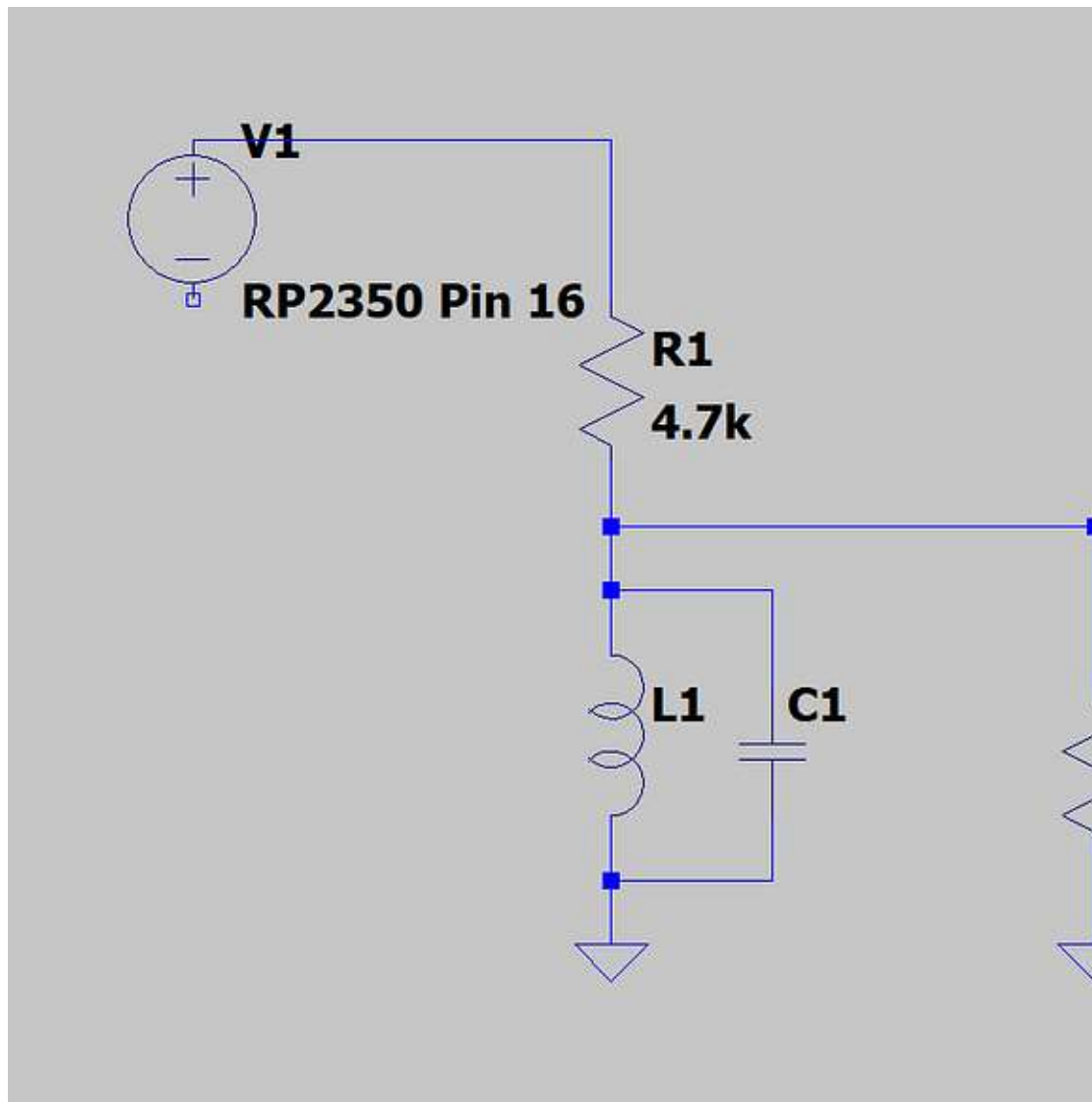
If we had used standard parts, we would know the inductance (L) of the coil and the capacitance (C) of the capacitor, and we could use a simple formula to calculate the resonant frequency (f):

We don't know the inductance of our hand-wound coil or the capacitance of our homemade capacitor.

The Simple Circuit

However, we have Python and our RP2350. We can build a device to tell us the resonant frequency.

We will use the RP2350's PWM output to simulate the antenna. We will use its ADC to simulate the earphone. We add three extra parts to our crystal radio to smooth out the signal (which the earphone coil normally does), and we have something that looks like this:
Press enter or click to view image in full size



Well, actually, it looks like this:
Press enter or click to view image in full size



The big red basket-weave coil is my hand-wound inductor. It is designed to have low self-capacitance and low self-coupling between coils, making it a high-performance part for our radio.

The black dial hides a commercial variable capacitor. I could replace it with two sheets of aluminium foil separated by a piece of waxed paper. But I got lazy.

The Code

As a nice break from our more ambitious projects, the code for the tester is fairly simple:

```
import machine
import time
import array

# --- Configuration ---
PWM_PIN_NUM = 16
ADC_PIN_NUM = 26

# --- Sweep Parameters ---
# For an AM radio coil, we'll sweep from 100 kHz to 2 MHz
START_FREQ = 100_000 # 100 kHz
END_FREQ = 2_000_000 # 2 MHz
NUM_STEPS = 100 # How many frequency steps to test
SETTLE_TIME_MS = 10 # How long to wait for the peak detector to stabilize

# --- Pin Setup ---
pwm = machine.PWM(machine.Pin(PWM_PIN_NUM))
adc = machine.ADC(machine.Pin(ADC_PIN_NUM))

# Set PWM duty cycle to 50%
pwm.duty_u16(32768)

buf = array.array('f', (0 for _ in range(NUM_STEPS)))

print("=" * 40)
print("Frequency Response Analyzer")
print(f"Sweeping from {START_FREQ/1000} kHz to {END_FREQ/1000} kHz in {NUM_STEPS} steps.")
print("=" * 40)

def main():
    max_adc_reading = -1
    resonant_frequency = 0
    step_size = (END_FREQ - START_FREQ) / NUM_STEPS

    try:
        for i in range(NUM_STEPS):
```

```

        current_freq = START_FREQ + i * step_size

        # 1. Set the stimulus frequency
        pwm.freq(int(current_freq))

        # 2. Wait for the peak detector's RC circuit to settle
        time.sleep_ms(SETTLE_TIME_MS)

        # 3. Read the response amplitude
        # Read a few times and average for a more stable
reading
        reading = 0
        for _ in range(100):
            reading += adc.read_u16()
        adc_reading = reading // 100
        buf[i] = 100 * adc_reading / 65535

        # 4. Check if this is the new peak
        if adc_reading > max_adc_reading:
            max_adc_reading = adc_reading
            resonant_frequency = current_freq

        # Optional: Print a progress bar
        progress = int((i / NUM_STEPS) * 30)
        print(f"[{'#' * progress}{'-' * (30 - progress)}]
{current_freq/1000:.1f} kHz -> {adc_reading}", end='\r')

    print("\n" + "="*40)
    print("Sweep Complete!")

    if max_adc_reading > 500: # Threshold to ensure a real peak
was found
        print(f"Peak detected at: {resonant_frequency/1000:.4f}
kHz")
        print(f"Peak ADC reading: {100 * max_adc_reading /
65535} %")
    else:
        print("No significant peak detected in the sweep
range.")
        print("Check wiring or adjust the START_FREQ/END_FREQ
range.")

    with open("sweep.out", "wb") as f:
        f.write(buf)

except KeyboardInterrupt:
    print("\nSweep stopped by user.")
finally:
    # Turn off the PWM to stop driving the circuit
    pwm.deinit()

main()

```

We set the PWM frequency at our start frequency. I have chosen the start and end frequencies to be reasonable for a coil destined for the AM band.

We wait a bit for the tank circuit to settle. Then, we take a number of ADC readings and average them for greater accuracy.

We keep track of the maximum voltage and the frequency at which we found it.

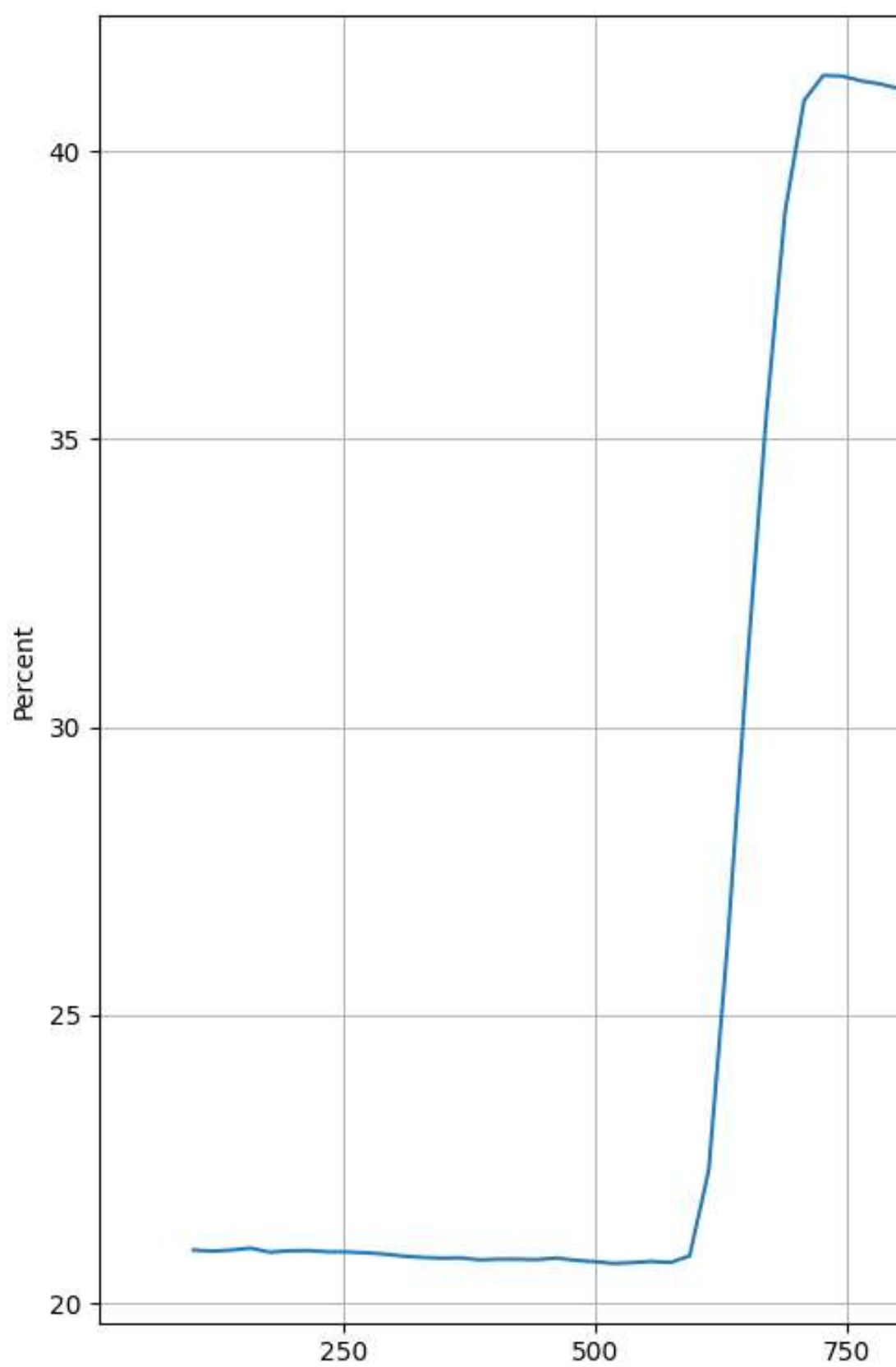
We show a progress bar, although we could probably have chosen values that make it fast enough not to need one. I was conservative on this first attempt.

Lastly, we print out the resonant frequency (where the least energy gets sucked off to ground).

```
> mpremote connect com4 run sweep.py
=====
Frequency Response Analyzer
Sweeping from 100.0 kHz to 2000.0 kHz in 100 steps.
=====
[#####-] 1981.0 kHz -> 30816
=====
Sweep Complete!
Peak detected at: 1582.0000 kHz
Peak ADC reading: 48.37262 %
```

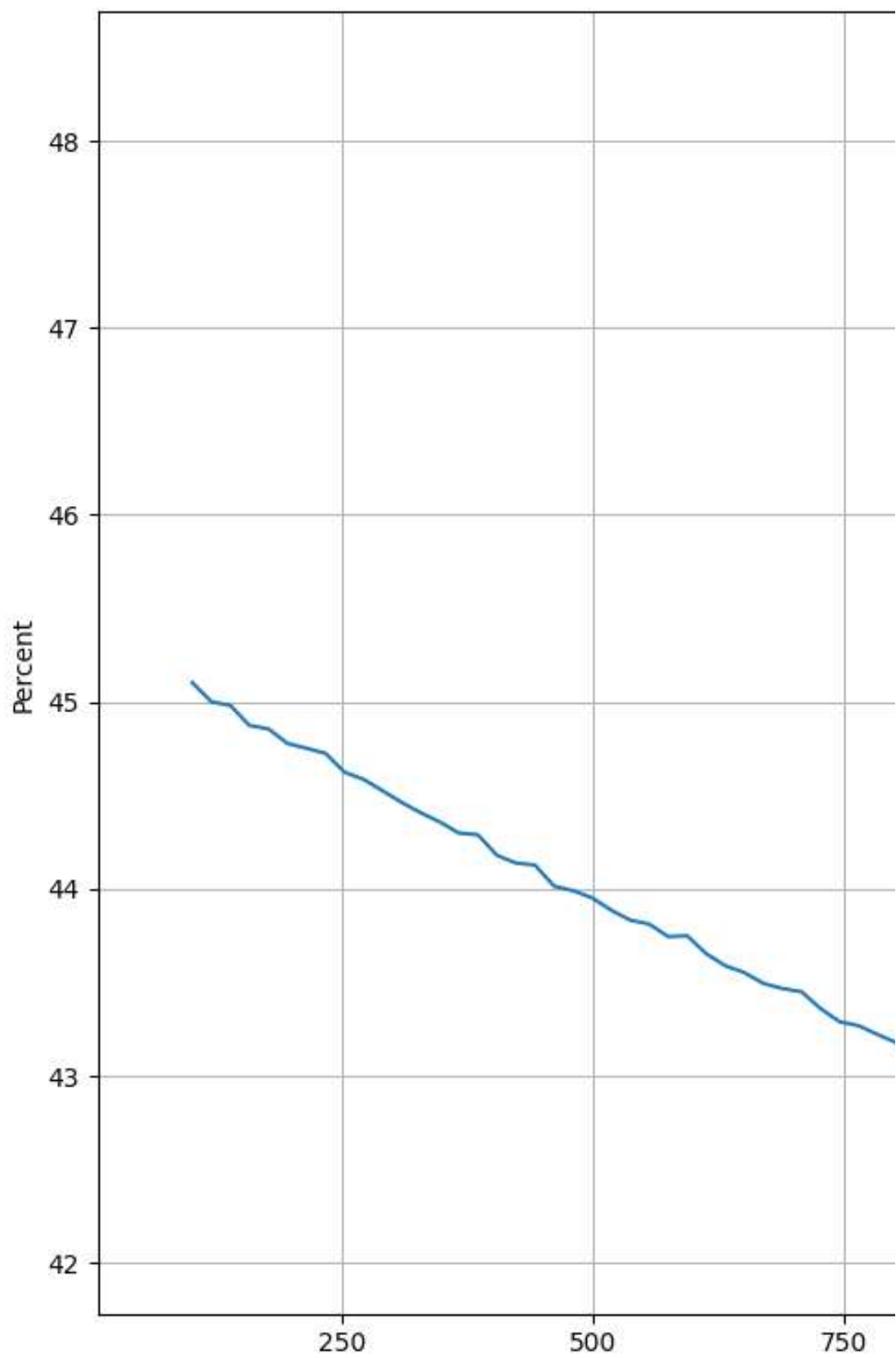
Along the way, I chose to collect the readings so I could show you this cool graph:

Press enter or click to view image in full size



That was with the variable capacitor set somewhat low. Here I set it higher:

Press enter or click to view image in full size



The program output looks like this:

```
> mpremote connect com4 run sweep.py
=====
Frequency Response Analyzer
Sweeping from 100.0 kHz to 2000.0 kHz in 100 steps.
=====
[#####-] 1981.0 kHz -> 30816
=====
Sweep Complete!
Peak detected at: 1582.0000 kHz
Peak ADC reading: 48.37262 %
```

I was quite pleased with how consistent the readings were. In radio work, I am used to values floating around (from noise, hand capacitance, hand-held probe resistance, etc.). But these were nicely repeatable every time. And I could nudge the variable capacitor a bit and see the reading go up or down just a bit, consistent with which way I turned the dial.

The Grapher

The program to show the plot is also quite simple:

```
import numpy as np
import matplotlib.pyplot as plt
import os
import array

def analyze_adc_capture():
    filename = 'sweep.out'
    if not os.path.exists(filename):
        print(f"Error: {filename} not found. Did you copy it from
the Pico?")
        return

    print(f"--- Analyzing '{filename}' ---")

    raw_data = np.fromfile(filename, dtype=np.float32)

    if raw_data.size == 0:
        print("File is empty.")
        return

    start = 100_000
    end = 2_000_000
    steps = raw_data.size
    step = int((end - start) / steps)
    # x_values = (f for f in range(start, end, step))
    x_values = array.array('f', (f / 1000 for f in range(start,
end, step)))
```

```

print(f"Number of samples: {raw_data.size}")

# --- Visualization ---
plt.figure(figsize=(12, 8))

# plt.subplot(2, 1, 1)
plt.title("Frequency Response")
plt.plot(x_values, raw_data)
plt.xlabel("Frequency kHz")
plt.ylabel("Percent")
# plt.xlim(0, 200)
plt.grid(True)

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    analyze_adc_capture()

```

I made a simple script to run the test, copy the file, and graph the result:

```

mpremote connect com4 run sweep.py
mpremote connect com4 cp :sweep.out sweep.out
graphit.py
> doit

> mpremote connect com4 run sweep.py
=====
Frequency Response Analyzer
Sweeping from 100.0 kHz to 2000.0 kHz in 100 steps.
=====
[#####-] 1981.0 kHz -> 30816
=====
Sweep Complete!
Peak detected at: 1582.0000 kHz
Peak ADC reading: 48.37262 %

> mpremote connect com4 cp :sweep.out sweep.out
cp :sweep.out sweep.out

> graphit.py
--- Analyzing 'sweep.out' ---
Number of samples: 100

```

Winding the Basket-Weave Coil

Winding coils on a cylindrical form is simple. Winding the basket-weave coil is a bit more complex, but the entire process took less than half an hour.

I started with a fat roll of blue painter's masking tape. It was 36 centimetres in circumference.

We need an odd number of poles to ensure that successive windings never overlap. I chose nine for convenience. Eleven is better, but going past seventeen is probably overkill.

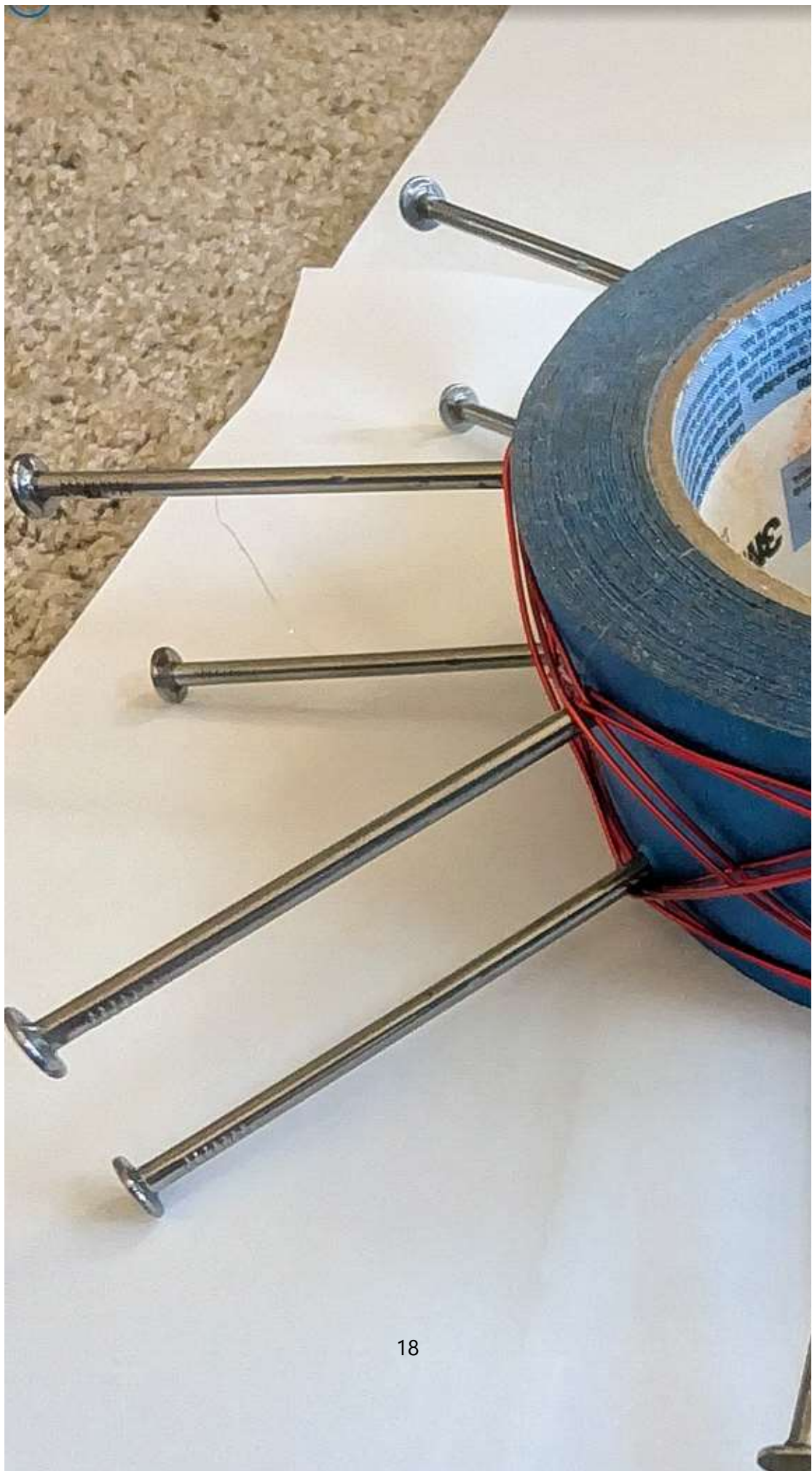
Nine goes into 36 four times, so I marked every four centimetres along the circumference of the tape roll.

Then I drilled two holes at each mark, a centimetre from each edge. I chose the drill bit to match a 10d nail.

Into each hole I pushed a 10-penny (10d) nail.

Then I started winding. I chose the “over two, under two” pattern, winding the enamel-coated copper wire over two nails, and then under the next two:

Press enter or click to view image in full size



Twenty minutes later, we have the coil:
Press enter or click to view image in full size



Now we run some glue over the wires where they cross in between the nails. When the glue sets, we pull out the nails and slide off the coil.

The last step is to sand off the insulation from the ends of the wire. I like to tin the ends with solder, since copper eventually oxidises, risking faulty contacts.

So, there we have two tools to make pretty basket-weave coils and test their tuning range. A coil form and a Python frequency response tester.

Now, go build yourself a radio.

Programming

Software Development

Radio